

Atelier

Mohamed Aziz Boualegui

2024-01-20

Intitulé du sujet

Analyse de transactions de cartes de crédit pour une détection précoce des défauts.

Descriptif du sujet

Cette étude se concentre sur l'analyse approfondie du comportement des détenteurs de cartes de crédit dans le but de détecter précocement les signes potentiels de défaut de paiement. En examinant les habitudes de dépenses, les modèles de remboursement, et d'autres comportements financiers, l'objectif est d'identifier des indicateurs prédictifs permettant d'évaluer le risque de défaut. L'objectif de cette recherche est de faciliter une approche proactive dans la gestion des défauts de paiement, offrant ainsi aux institutions financières des solutions plus précises et rapides pour évaluer et atténuer les risques inhérents aux transactions par cartes de crédit.

Travail à réaliser

- Pré-traitement des données (Valeurs atypiques, valeurs manquantes, analyse de la qualité des données)
- L'analyse exploratoire des données (Analyse univariée, bivariée, Visualisation des données, ..)
- Transformation des variables (normalisation, discrétisation, ...)
- Analyse de corrélation, analyse de variance, test d'Indépendance
- Tableau de bord (dashboard) avec Rshiny.

Informations sur La Dataset:

La Dataset contient des informations sur les paiements par défaut, les facteurs démographiques, les données de crédit, l'historique des paiements et les relevés de factures des clients de cartes de crédit à Taiwan d'avril 2005 à septembre 2005.

```
#Importation des Librairies
```

```
library(ggplot2)
library(dplyr)
library(ggthemes)
```

```
## Warning: package 'ggthemes' was built under R version 4.3.2
```

```
library(corrplot)
```

```
## Warning: package 'corrplot' was built under R version 4.3.2
```

```
library(randomForest)
```

```
## Warning: package 'randomForest' was built under R version 4.3.2
```

```
library(scorecard)
```

```
## Warning: package 'scorecard' was built under R version 4.3.2
```

Importation des données

```
#Lecture du fichier csv
```

```
data <- read.csv("C:\\Users\\Kamel\\OneDrive\\Documents\\ESSAIT 1\\Documents\\Atelier\\Donnees\\Data.csv")  
head(data)
```

```
##   ID LIMIT_BAL SEX EDUCATION MARRIAGE AGE PAY_0 PAY_2 PAY_3 PAY_4 PAY_5 PAY_6  
## 1 1    20000  2      2        1  24    2      2    -1    -1    -2    -2  
## 2 2    120000  2      2        2  26   -1     2     0     0     0     2  
## 3 3     90000  2      2        2  34    0     0     0     0     0     0  
## 4 4     50000  2      2        1  37    0     0     0     0     0     0  
## 5 5     50000  1      2        1  57   -1     0    -1     0     0     0  
## 6 6     50000  1      1        2  37    0     0     0     0     0     0  
##   BILL_AMT1 BILL_AMT2 BILL_AMT3 BILL_AMT4 BILL_AMT5 BILL_AMT6 PAY_AMT1 PAY_AMT2  
## 1      3913      3102      689         0         0         0         0      689  
## 2      2682      1725      2682      3272      3455      3261         0      1000  
## 3     29239     14027     13559     14331     14948     15549     1518     1500  
## 4     46990     48233     49291     28314     28959     29547     2000     2019  
## 5      8617      5670     35835     20940     19146     19131     2000     36681  
## 6     64400     57069     57608     19394     19619     20024     2500     1815  
##   PAY_AMT3 PAY_AMT4 PAY_AMT5 PAY_AMT6 default.payment.next.month  
## 1         0         0         0         0                             1  
## 2        1000        1000         0        2000                             1  
## 3        1000        1000        1000        5000                             0  
## 4        1200        1100        1069        1000                             0  
## 5       10000        9000         689         679                             0  
## 6         657        1000        1000         800                             0
```

Contenu

- ID: ID de chaque client
- LIMIT_BAL: Montant du crédit accordé en dollars NT (comprend le crédit individuel et familial/supplémentaire)
- SEX: Gendre (1=male, 2=femelle)
- EDUCATION: (1=graduate school, 2=university, 3=high school, 4=others, 5=unknown, 6=unknown)
- AGE: Age en années
- PAY_0: Statut de remboursement en septembre 2005 (-1=payer dûment, 1=retard de paiement d'un mois, 2=retard de paiement de deux mois, ... 8=retard de paiement de huit mois, 9=retard de paiement de neuf mois et plus)
- PAY_2: Statut de remboursement en août 2005 (échelle identique à celle ci-dessus)
- PAY_3: Statut de remboursement en Juillet 2005 (échelle identique à celle ci-dessus)

- PAY_4: Statut de remboursement en Juin 2005 (échelle identique à celle ci-dessus)
- PAY_5: Statut de remboursement en May 2005 (échelle identique à celle ci-dessus)
- PAY_6: Statut de remboursement en Avril 2005 (échelle identique à celle ci-dessus)
- BILL_AMT1: Montant du relevé de facture en septembre 2005 (dollar NT)
- BILL_AMT2: Montant du relevé de facture en août 2005 (dollar NT)
- BILL_AMT3: Montant du relevé de facture en Juillet 2005 (dollar NT)
- BILL_AMT4: Montant du relevé de facture en Juin 2005 (dollar NT)
- BILL_AMT5: Montant du relevé de facture en May 2005 (dollar NT)
- BILL_AMT6: Montant du relevé de facture en Avril 2005 (dollar NT)
- PAY_AMT1: Montant du paiement précédent en septembre 2005 (NT dollar)
- PAY_AMT2: Montant du paiement précédent en août 2005 (NT dollar)
- PAY_AMT3: Montant du paiement précédent en Juillet 2005 (NT dollar)
- PAY_AMT4: Montant du paiement précédent en Juin 2005 (NT dollar)
- PAY_AMT5: Montant du paiement précédent en May 2005 (NT dollar)
- PAY_AMT6: Montant du paiement précédent en Avril 2005 (NT dollar)
- default.payment.next.month: Default payment (1=yes, 0=no)

```
summary(data)
```

```
##          ID          LIMIT_BAL          SEX          EDUCATION
## Min.      :    1      Min.      : 10000      Min.      :1.000      Min.      :0.000
## 1st Qu.: 7501      1st Qu.:  50000      1st Qu.:1.000      1st Qu.:1.000
## Median :15000      Median : 140000      Median :2.000      Median :2.000
## Mean     :15000      Mean     : 167484      Mean     :1.604      Mean     :1.853
## 3rd Qu.:22500      3rd Qu.: 240000      3rd Qu.:2.000      3rd Qu.:2.000
## Max.     :30000      Max.     :1000000      Max.     :2.000      Max.     :6.000
##    MARRIAGE          AGE          PAY_0          PAY_2
## Min.      :0.000      Min.      :21.00      Min.      : -2.0000      Min.      : -2.0000
## 1st Qu.:1.000      1st Qu.:28.00      1st Qu.: -1.0000      1st Qu.: -1.0000
## Median :2.000      Median :34.00      Median : 0.0000      Median : 0.0000
## Mean     :1.552      Mean     :35.49      Mean     : -0.0167      Mean     : -0.1338
## 3rd Qu.:2.000      3rd Qu.:41.00      3rd Qu.: 0.0000      3rd Qu.: 0.0000
## Max.     :3.000      Max.     :79.00      Max.     : 8.0000      Max.     : 8.0000
##    PAY_3          PAY_4          PAY_5          PAY_6
## Min.      : -2.0000      Min.      : -2.0000      Min.      : -2.0000      Min.      : -2.0000
## 1st Qu.: -1.0000      1st Qu.: -1.0000      1st Qu.: -1.0000      1st Qu.: -1.0000
## Median : 0.0000      Median : 0.0000      Median : 0.0000      Median : 0.0000
## Mean     : -0.1662      Mean     : -0.2207      Mean     : -0.2662      Mean     : -0.2911
## 3rd Qu.: 0.0000      3rd Qu.: 0.0000      3rd Qu.: 0.0000      3rd Qu.: 0.0000
## Max.     : 8.0000      Max.     : 8.0000      Max.     : 8.0000      Max.     : 8.0000
##    BILL_AMT1      BILL_AMT2      BILL_AMT3      BILL_AMT4
## Min.      : -165580      Min.      : -69777      Min.      : -157264      Min.      : -170000
## 1st Qu.:   3559      1st Qu.:   2985      1st Qu.:   2666      1st Qu.:   2327
## Median :  22382      Median :  21200      Median :  20089      Median :  19052
## Mean     :  51223      Mean     :  49179      Mean     :  47013      Mean     :  43263
## 3rd Qu.:  67091      3rd Qu.:  64006      3rd Qu.:  60165      3rd Qu.:  54506
## Max.     : 964511      Max.     : 983931      Max.     :1664089      Max.     : 891586
##    BILL_AMT5      BILL_AMT6      PAY_AMT1      PAY_AMT2
## Min.      : -81334      Min.      : -339603      Min.      :    0      Min.      :    0
## 1st Qu.:   1763      1st Qu.:   1256      1st Qu.:  1000      1st Qu.:   833
## Median :  18105      Median :  17071      Median :  2100      Median :   2009
## Mean     :  40311      Mean     :  38872      Mean     :  5664      Mean     :   5921
## 3rd Qu.:  50191      3rd Qu.:  49198      3rd Qu.:  5006      3rd Qu.:   5000
## Max.     : 927171      Max.     : 961664      Max.     :873552      Max.     :1684259
```

```
##      PAY_AMT3      PAY_AMT4      PAY_AMT5      PAY_AMT6
## Min.   :      0   Min.   :      0   Min.   :      0.0   Min.   :      0.0
## 1st Qu.:    390   1st Qu.:    296   1st Qu.:    252.5   1st Qu.:    117.8
## Median :   1800   Median :   1500   Median :   1500.0   Median :   1500.0
## Mean   :   5226   Mean   :   4826   Mean   :   4799.4   Mean   :   5215.5
## 3rd Qu.:   4505   3rd Qu.:   4013   3rd Qu.:   4031.5   3rd Qu.:   4000.0
## Max.   : 896040   Max.   : 621000   Max.   : 426529.0   Max.   : 528666.0
## default.payment.next.month
## Min.   :0.0000
## 1st Qu.:0.0000
## Median :0.0000
## Mean   :0.2212
## 3rd Qu.:0.0000
## Max.   :1.0000
```

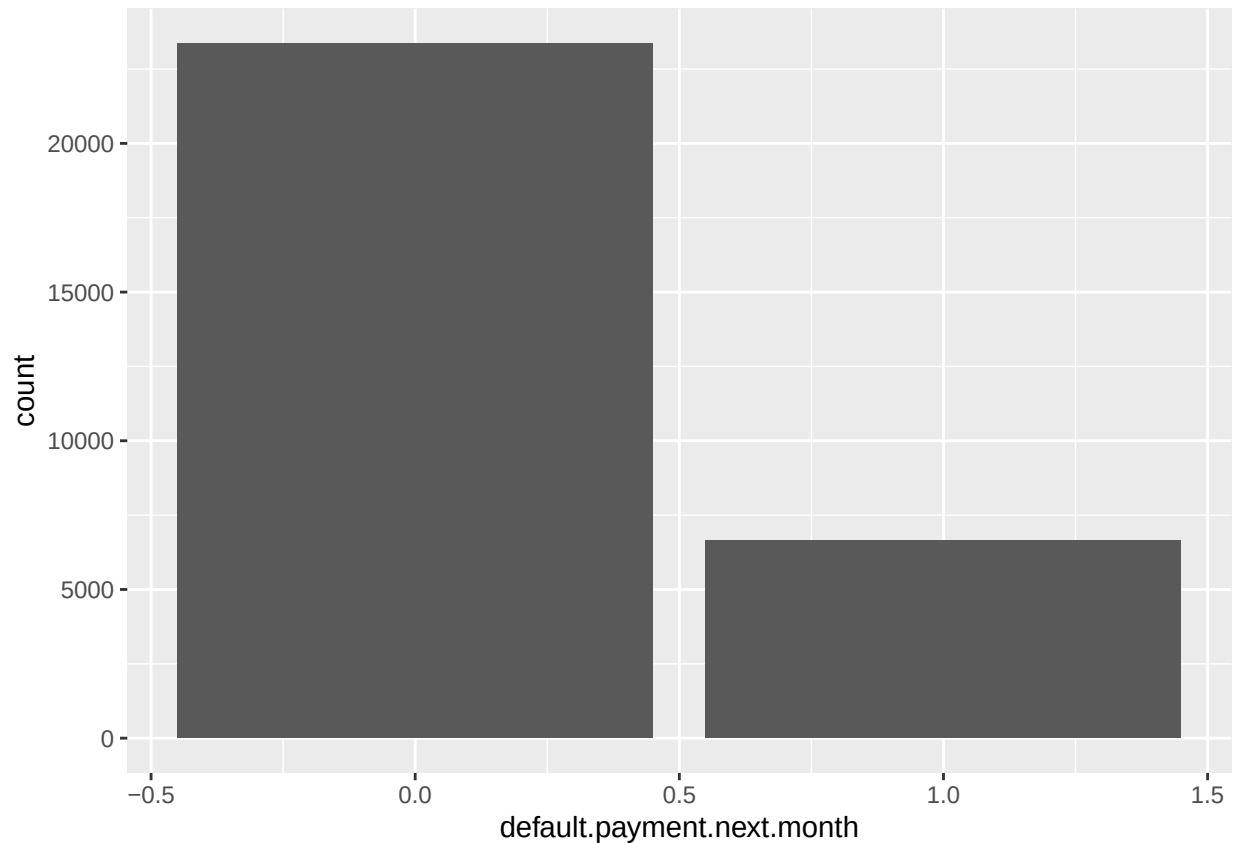
```
data <- subset(data, select = -ID)
```

Definition de variables qualitatives

```
factor_variables <- c("SEX", "EDUCATION", "MARRIAGE")
quantitative_variables <- c("LIMIT_BAL", "AGE", "PAY_0", "PAY_2", "PAY_3", "PAY_4", "PAY_5", "PAY_6", "PAY_7")
data[factor_variables] <- lapply(data[factor_variables], function(x) as.factor(x))
target <- "default.payment.next.month"
```

Equilibre des donnees

```
ggplot(data, aes(default.payment.next.month)) +
  geom_bar()
```



```
percent <- data %>%
  count(default.payment.next.month == 1)*100/30000
percent
```

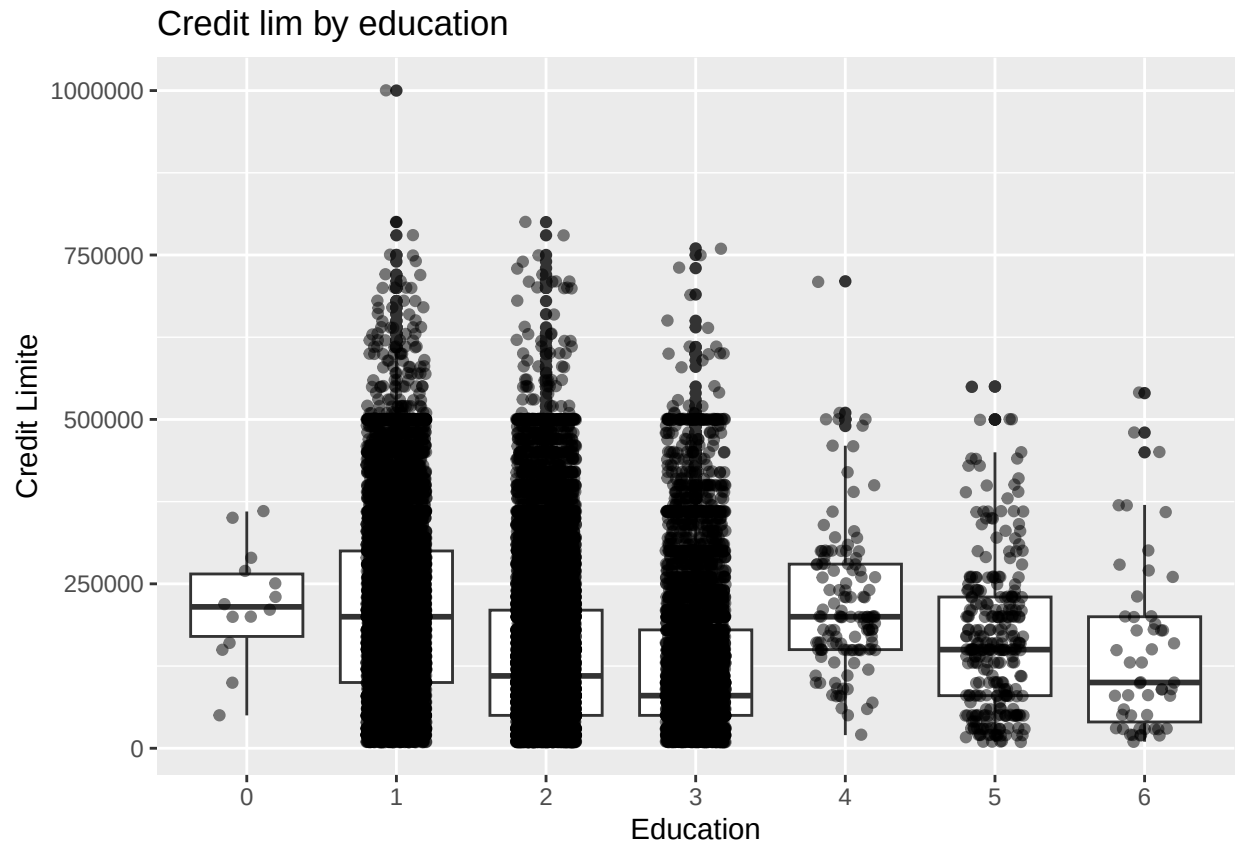
```
## default.payment.next.month == 1      n
## 1                0.000000000 77.88
## 2                0.003333333 22.12
```

Les données ne sont pas trop inéquilibrées, 22% des clients ont commis des défauts alors que 78% n'ont pas.

Boxplot credit lim by education

```
credit_lim_by_education_bp <- data %>%
  ggplot(aes(EDUCATION, LIMIT_BAL), fill = EDUCATION) +
  geom_boxplot() +
  geom_jitter(position = position_jitter(width = 0.2), alpha = 0.5) +
  labs(title = "Credit lim by education", x = "Education", y = "Credit Limite")

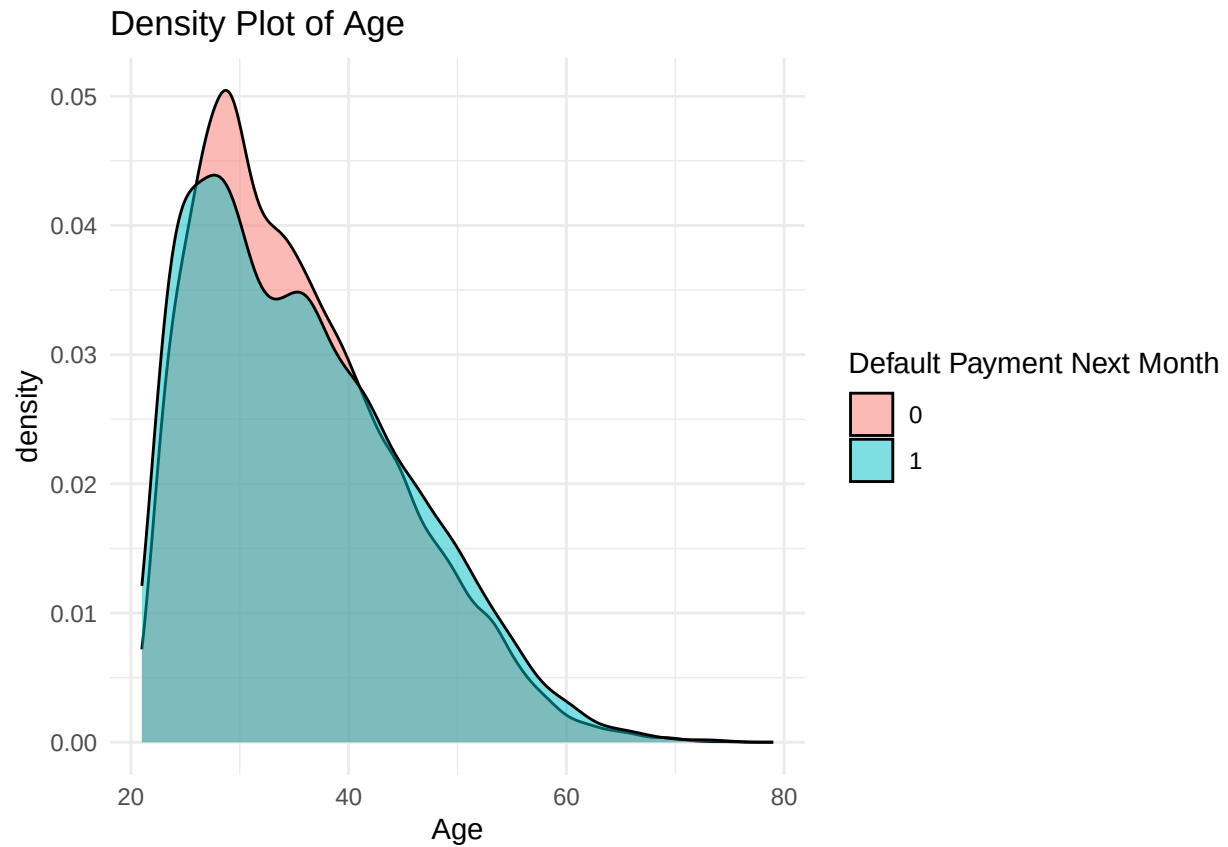
credit_lim_by_education_bp
```



On remarque l'existence de la valeur 0 qui n'est pas définie sur la Dataset, on remarque aussi que les classes d'éducation graduate school, university et high school sont les plus présents dans notre Dataset.

Creation de density plot par age

```
density_plot_Age <- data %>%
  ggplot(aes(x = AGE, fill = factor(default.payment.next.month))) +
  geom_density(alpha = 0.5) +
  labs(title = "Density Plot of Age",
       x = "Age",
       fill = "Default Payment Next Month") +
  theme_minimal()
density_plot_Age
```

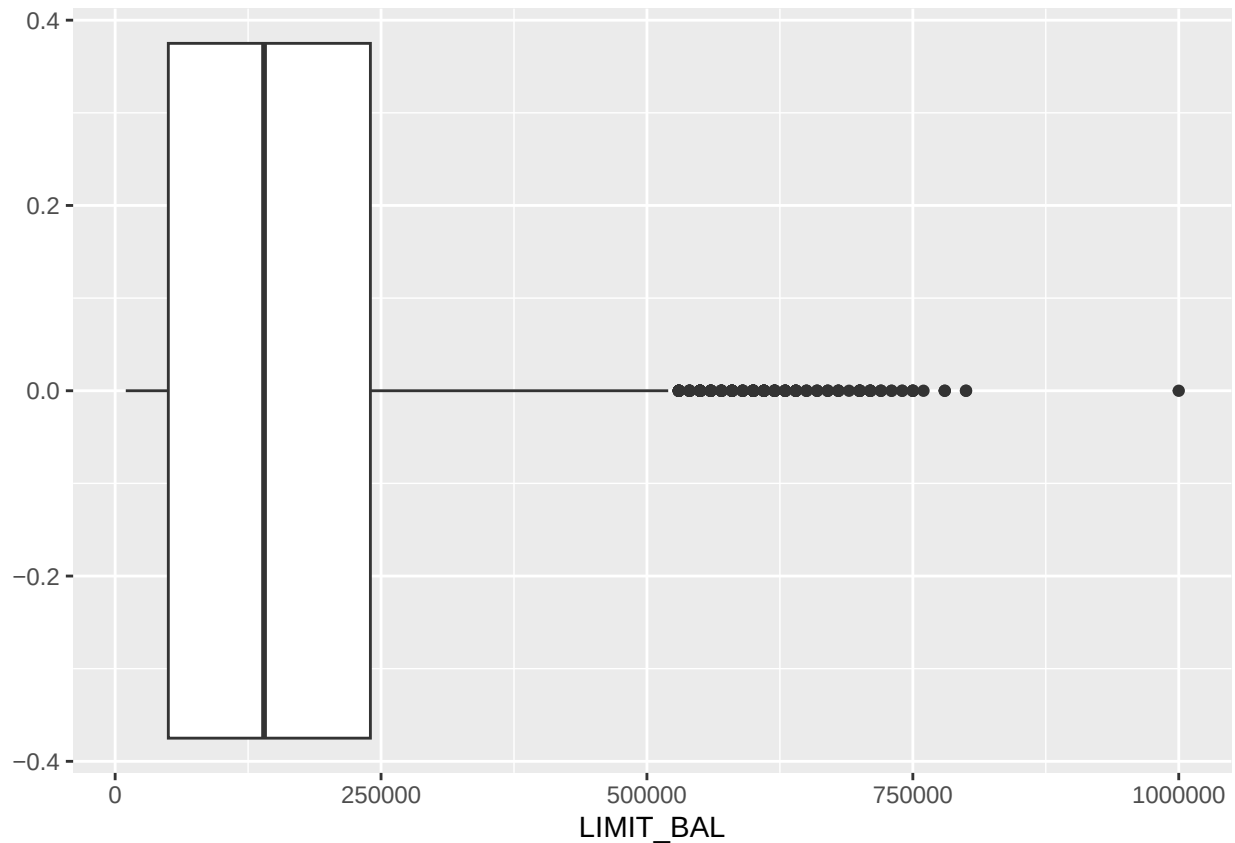


Les ages des clients est superieur à 20, aux alentours de 30 ans, il'y a plus de des clients qui n'ont pas faits des défauts, ce qui n'est pas le cas pour les agés de 40 ans et plus.

Creation de boxplot de LIMIT_BAL

```
boxplot_limit_bal <- data %>%  
  ggplot(aes(x = LIMIT_BAL)) +  
  geom_boxplot()
```

```
boxplot_limit_bal
```



```
q <- quantile(data$LIMIT_BAL, probs = c(0.25, 0.5, 0.75))
q
```

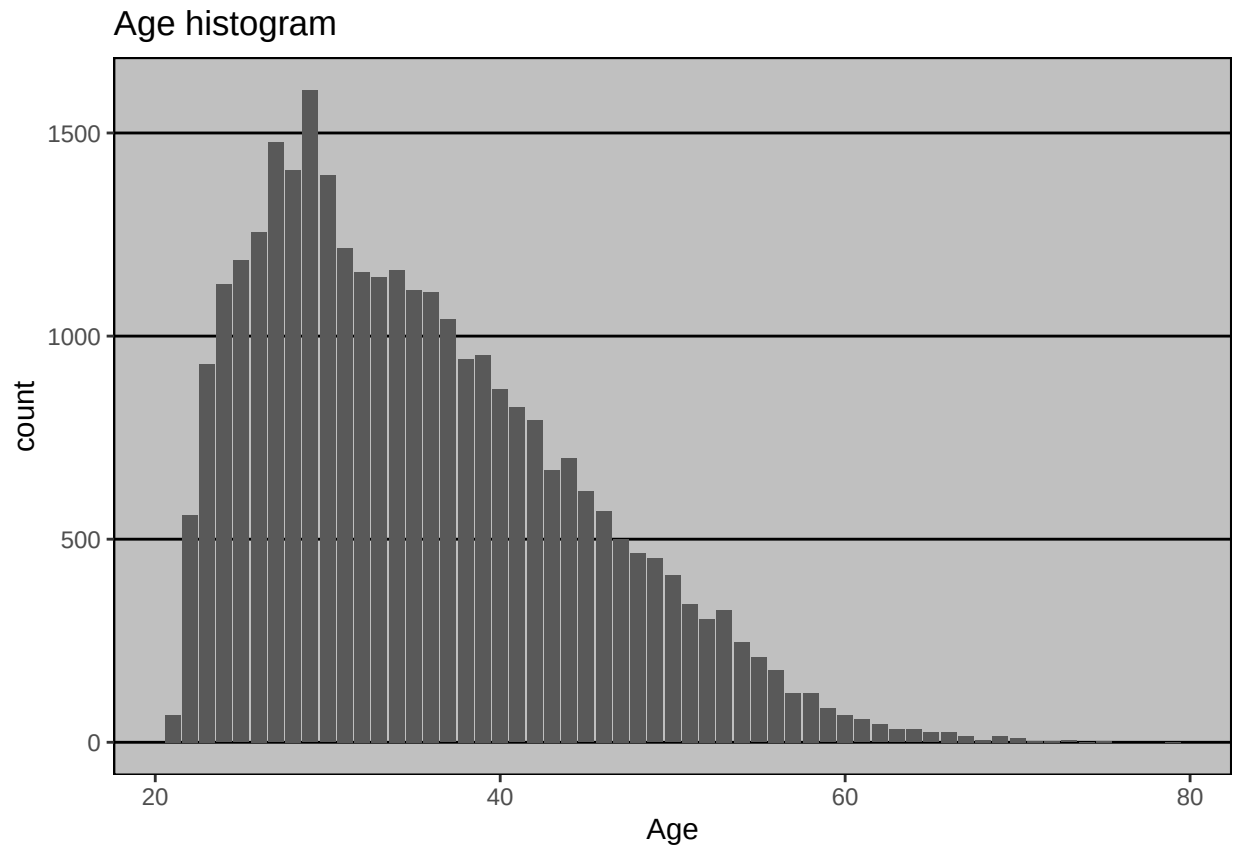
```
##      25%      50%      75%
##  50000 140000 240000
```

La valeur de plus part des crédits est aux alentours de 50,000NTD and 240,000NTD comme on peut voir, avec la presence de multiples valeurs aberrantes superieurs a 500,000NTD

Creation de histogramme d'Age

```
Age_hist <- ggplot(data, aes(AGE), fill = default.payment.next.month) +
  geom_bar() +
  labs(title = "Age histogram", x = "Age") +
  theme_excel()

Age_hist
```



Les ages de la plus part des clients est aux alentours de 30 ans, de plus la distribution semble suivre la loi normal, faisons un test Shapiro

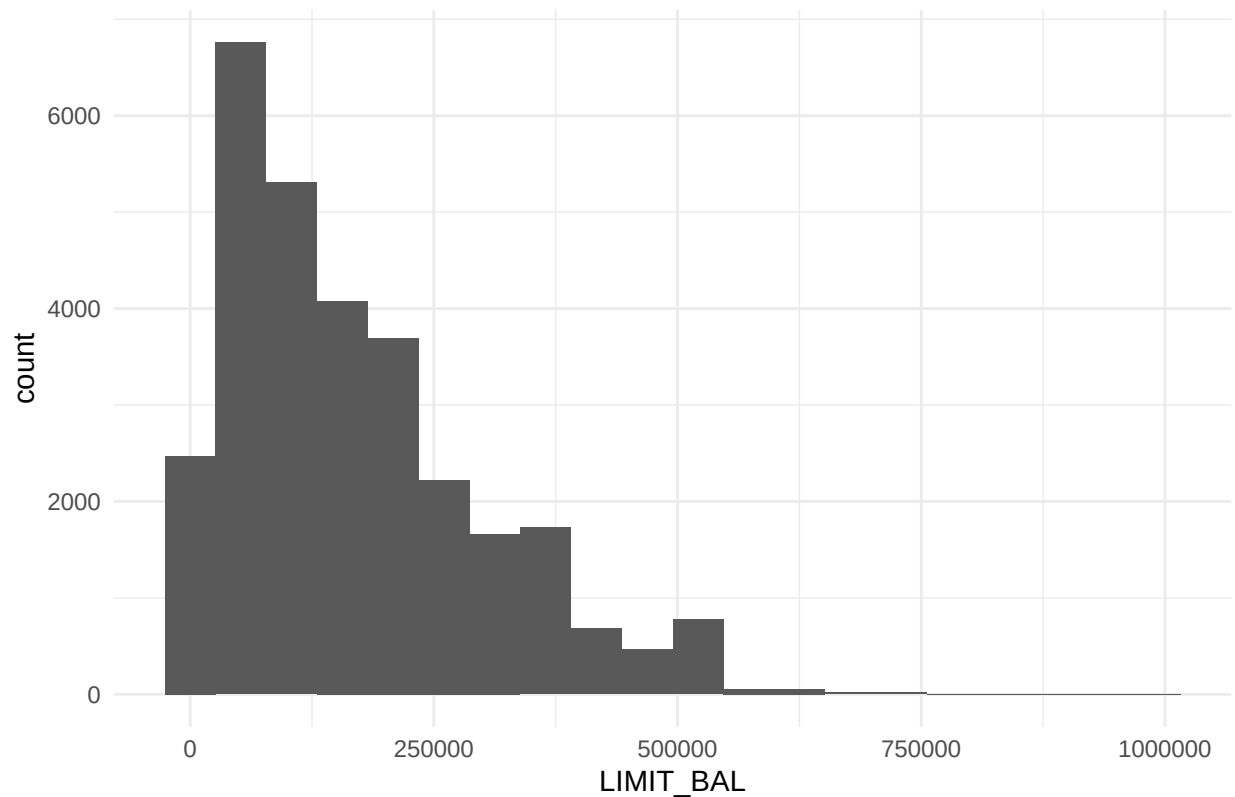
```
sample_data <- sample(data$AGE, 5000)
normality <- shapiro.test(sample_data)
normality
```

```
##
##  Shapiro-Wilk normality test
##
## data:  sample_data
## W = 0.9516, p-value < 2.2e-16
```

Paiement LIMIT_BAL

```
LIMIT_BAL_hist <- data %>%
  ggplot(aes(LIMIT_BAL)) +
  geom_histogram(bins = 20) +
  labs(title = "Histogramme LIMIT_BAL") +
  theme_minimal()
LIMIT_BAL_hist
```

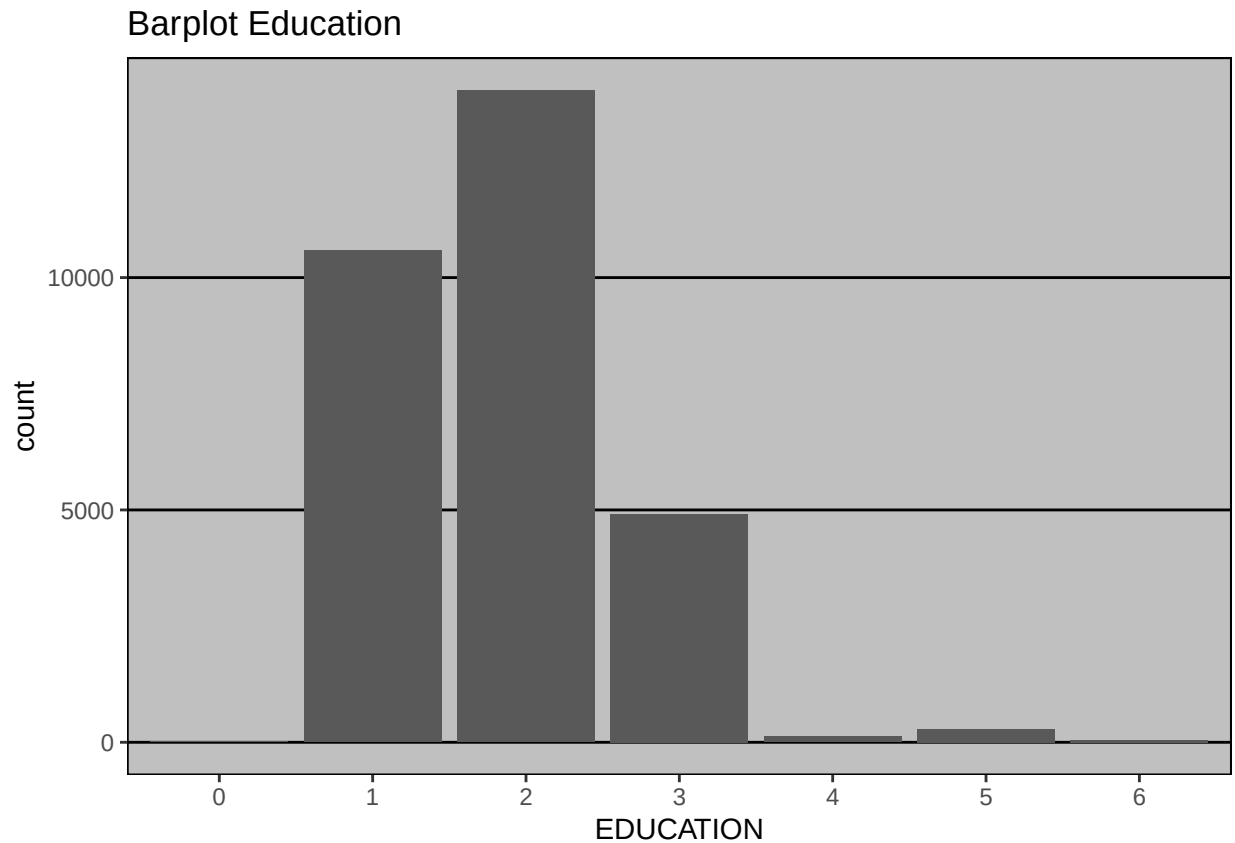
Histogramme LIMIT_BAL



Relation entre les défauts et l'éducation

```
suppressWarnings({def_edu <- data %>%
  ggplot(aes(EDUCATION, fill = default.payment.next.month)) +
  geom_bar() +
  labs(title = "Barplot Education") +
  theme_excel()
def_edu})
```

```
## Warning: The following aesthetics were dropped during statistical transformation: fill
## i This can happen when ggplot fails to infer the correct grouping structure in
## the data.
## i Did you forget to specify a 'group' aesthetic or to convert a numerical
## variable into a factor?
```

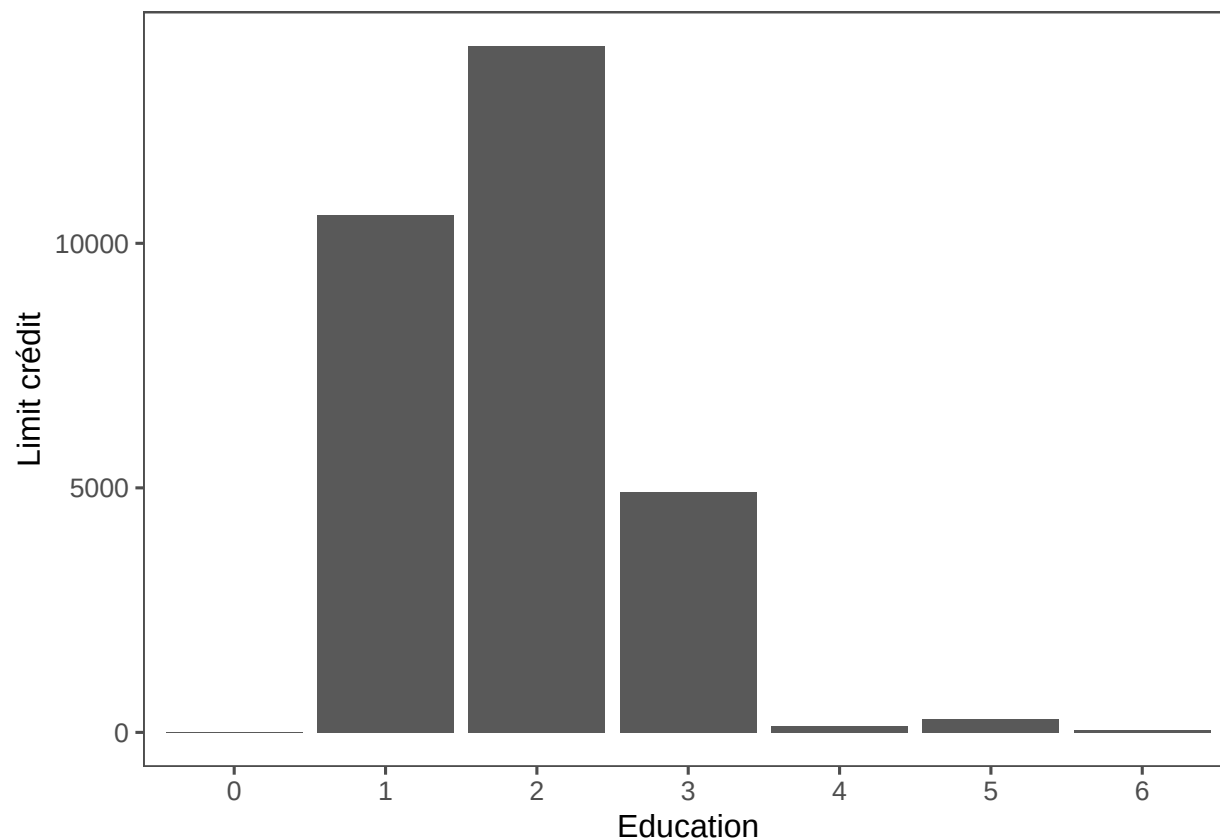


On remarque que, plus le niveau d'éducation est haut, il ya plus de chance que le client commit un défaut.

Relation entre LIMIT_BAL et EDUCATION

```
LIM_EDU <- data %>%
  ggplot( aes(x = EDUCATION, fill = LIMIT_BAL)) +
  geom_bar() +
  labs(x = 'Education') +
  labs(y = 'Limit crédit') +
  theme_few()
LIM_EDU
```

```
## Warning: The following aesthetics were dropped during statistical transformation: fill
## i This can happen when ggplot fails to infer the correct grouping structure in
##   the data.
## i Did you forget to specify a 'group' aesthetic or to convert a numerical
##   variable into a factor?
```



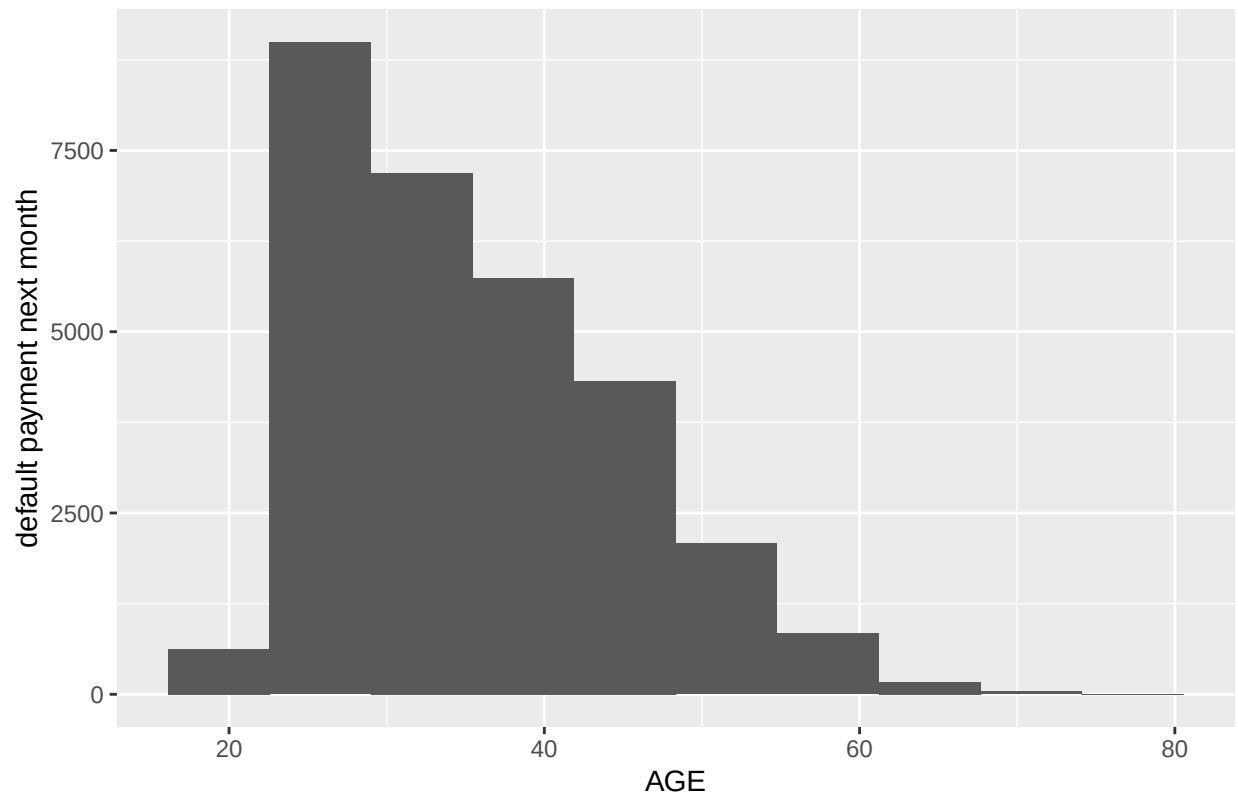
Il est clair que le seuil des montants des crédits accordés aux étudiants universitaires est plus haut que celui des autres étudiants.

Relation entre âge et défaut

```
age_def <- data %>%
  ggplot(aes(AGE, fill = default.payment.next.month)) +
  geom_histogram(bins = 10) +
  labs(title = "Relation entre âge et défaut", y = "default payment next month")
age_def
```

```
## Warning: The following aesthetics were dropped during statistical transformation: fill
## i This can happen when ggplot fails to infer the correct grouping structure in
##   the data.
## i Did you forget to specify a 'group' aesthetic or to convert a numerical
##   variable into a factor?
```

Relation entre age et défaut

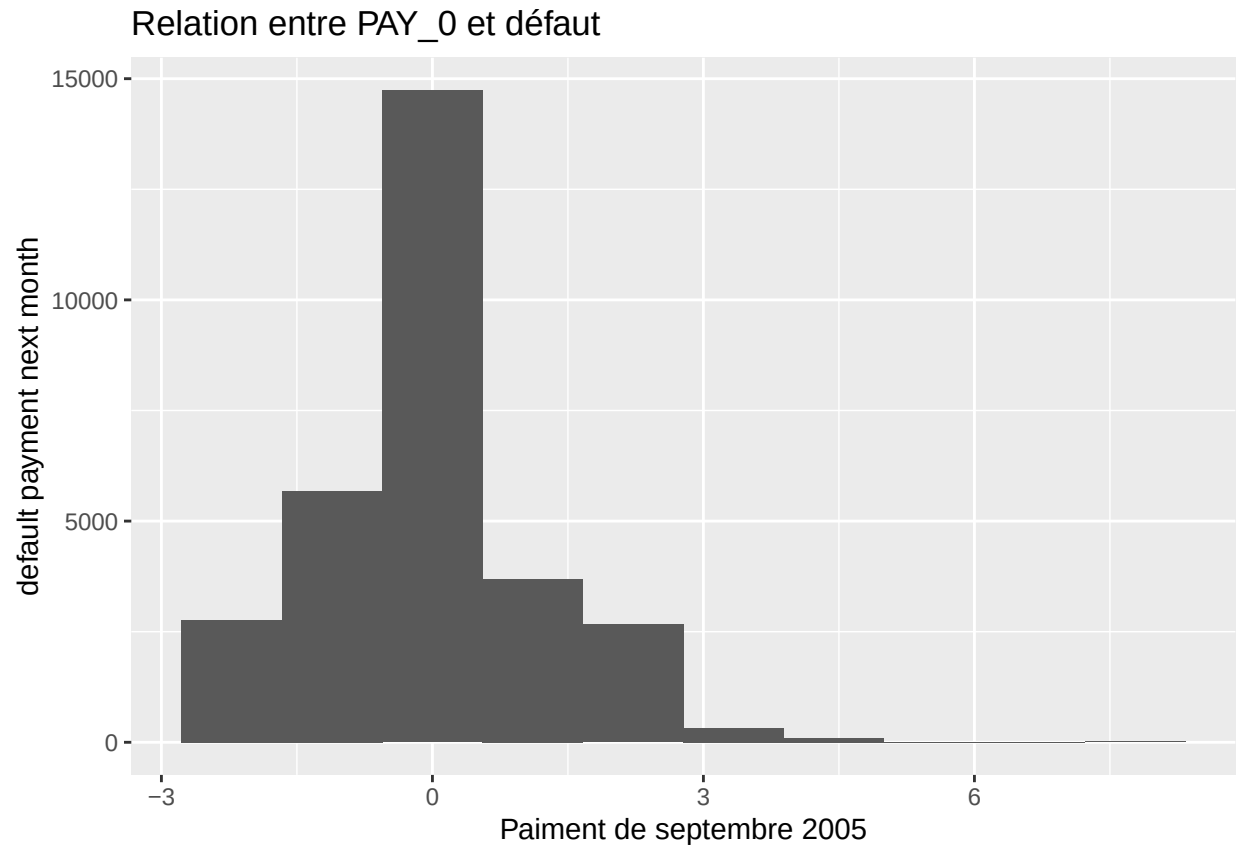


les valeurs des défauts sont plus hautes pour les alentours de 30 ans

Exploration des relations des PAY, PAY_ATM, et BILL avec défaut

```
graph_PAY <- data %>%  
  ggplot(aes(PAY_0, fill = default.payment.next.month)) +  
  geom_histogram(bins = 10) +  
  labs(title = "Relation entre PAY_0 et défaut", x = "Paiement de septembre 2005", y = "default payment n  
  
graph_PAY
```

```
## Warning: The following aesthetics were dropped during statistical transformation: fill  
## i This can happen when ggplot fails to infer the correct grouping structure in  
## the data.  
## i Did you forget to specify a 'group' aesthetic or to convert a numerical  
## variable into a factor?
```

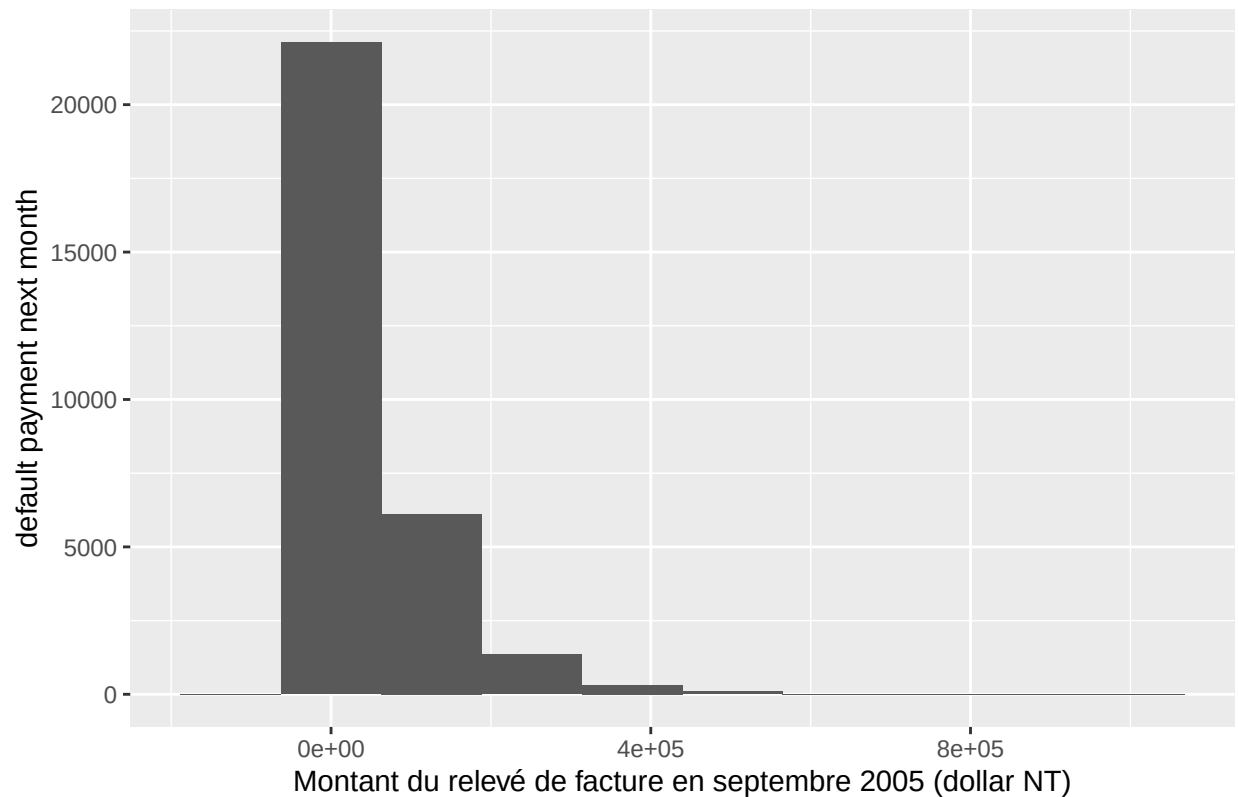


```
graph_BILL <- data %>%
  ggplot(aes(BILL_AMT1, fill = default.payment.next.month)) +
  geom_histogram(bins = 10) +
  labs(title = "Relation entre BILL_AMT1 et défaut", x = "Montant du relevé de facture en septembre 2005")

graph_BILL
```

```
## Warning: The following aesthetics were dropped during statistical transformation: fill
## i This can happen when ggplot fails to infer the correct grouping structure in
## the data.
## i Did you forget to specify a 'group' aesthetic or to convert a numerical
## variable into a factor?
```

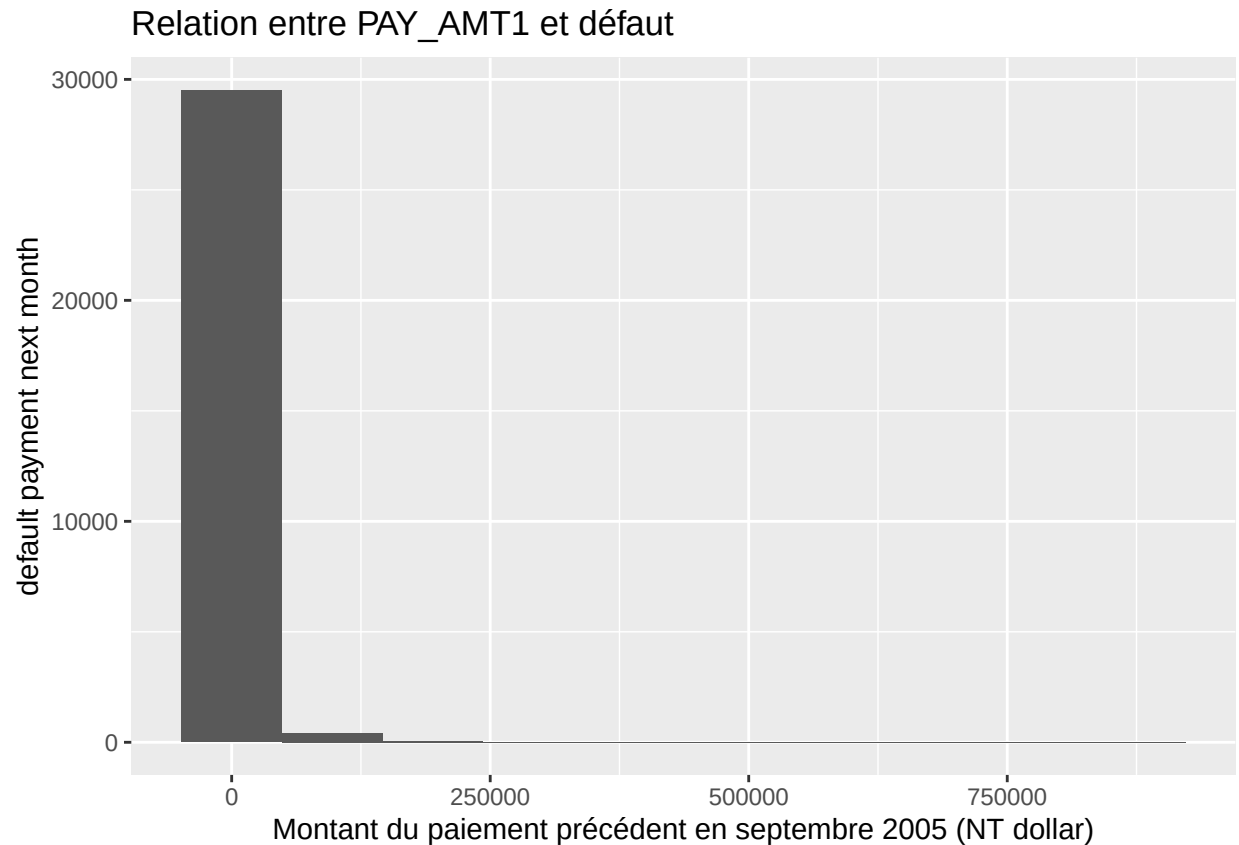
Relation entre BILL_AMT1 et défaut



```
graph_PAY_ATM <- data %>%
  ggplot(aes(PAY_AMT1, fill = default.payment.next.month)) +
  geom_histogram(bins = 10) +
  labs(title = "Relation entre PAY_AMT1 et défaut", x = "Montant du paiement précédent en septembre 2005")

graph_PAY_ATM
```

```
## Warning: The following aesthetics were dropped during statistical transformation: fill
## i This can happen when ggplot fails to infer the correct grouping structure in
##   the data.
## i Did you forget to specify a 'group' aesthetic or to convert a numerical
##   variable into a factor?
```



L'existence de la valeur 0, pas définie dans la Dataset pose un problem.

Teste de Khi2

```
# Define your target variable
target_variable <- data$default.payment.next.month # Replace 'target_variable_name_here' with your actual variable name

# Create an empty dataframe to store results
chi_squared_results <- data.frame(
  Factor_Variable = character(), # Column to store factor variable names
  ChiSquared = numeric(),        # Column to store chi-squared values
  DF = numeric(),               # Column to store degrees of freedom
  p_value = numeric()          # Column to store p-values
)

# Function to perform chi-squared test for each factor variable
for (col in names(data)) {
  if (is.factor(data[[col]]) && col != "default.payment.next.month") {
    contingency_table <- table(data[[col]], target_variable)
    chi_squared_test <- chisq.test(contingency_table)

    # Append results to dataframe
    chi_squared_results <- rbind(chi_squared_results, data.frame(
      Factor_Variable = col,
      ChiSquared = chi_squared_test$statistic,
```

```

    DF = chi_squared_test$parameter,
    p_value = chi_squared_test$p.value
  ))
}
}

```

```

## Warning in chisq.test(contingency_table): Chi-squared approximation may be
## incorrect

```

```

# Print the dataframe with chi-squared test results
chi_squared_results

```

```

##           Factor_Variable ChiSquared DF      p_value
## X-squared           SEX      47.7088  1 4.944679e-12
## X-squared1      EDUCATION    163.2166  6 1.233263e-32
## X-squared2      MARRIAGE     35.6624  3 8.825862e-08

```

```

###Teste de V de Cramer

```

```

# Load the vcd package
suppressWarnings(library(vcd))

```

```

## Loading required package: grid

```

```

compute_cramers_v <- function(data) {
  # Initialize an empty matrix to store Cramer's V values
  cramer_v_matrix <- matrix(NA, ncol(data), ncol(data))
  colnames(cramer_v_matrix) <- names(data)
  rownames(cramer_v_matrix) <- names(data)

  # Iterate over all pairs of variables
  for (i in 1:ncol(data)) {
    for (j in 1:ncol(data)) {
      # Compute Cramer's V if both variables are factors
      if (is.factor(data[[i]]) && is.factor(data[[j]])) {
        # Create a contingency table
        contingency_table <- table(data[[i]], data[[j]])

        # Compute Cramer's V
        assoc_stats <- assocstats(contingency_table)
        cramer_v_matrix[i, j] <- assoc_stats$cramer
      } else {
        cramer_v_matrix[i, j] <- NA # Assign NA if variables are not both factors
      }
    }
  }

  return(cramer_v_matrix)
}

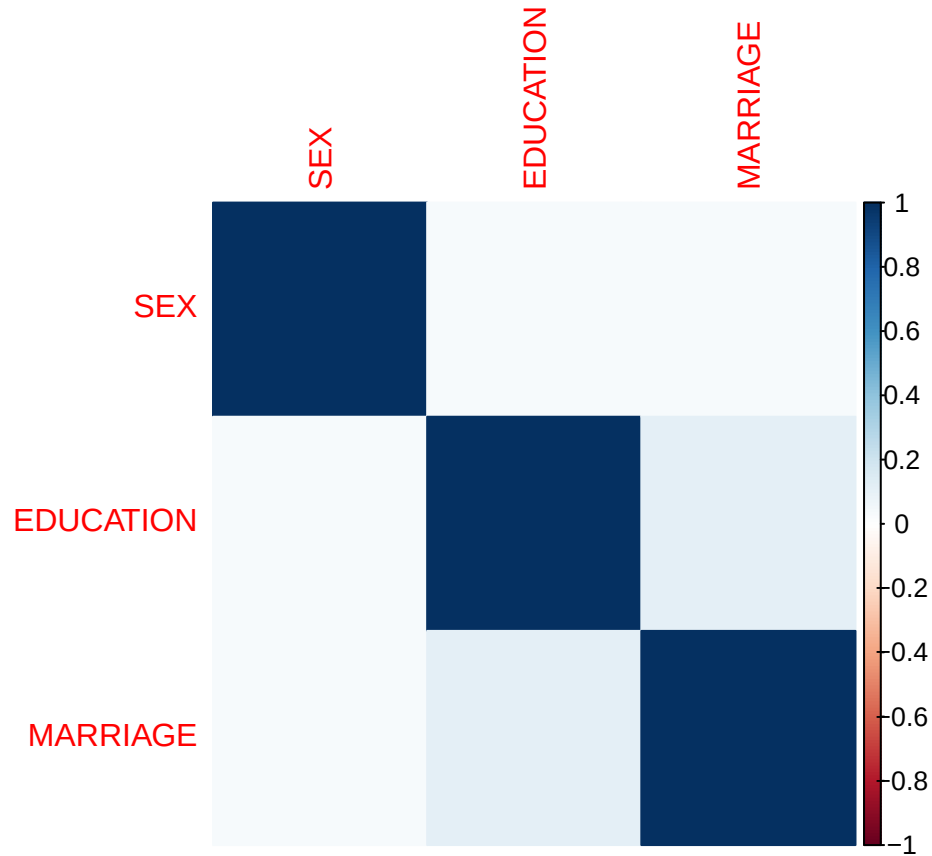
```

```

# Compute Cramer's V for all pairs of variables

```

```
cramer_v_matrix <- compute_cramers_v(data[factor_variables])
corrplot(cramer_v_matrix, method = "color")
```

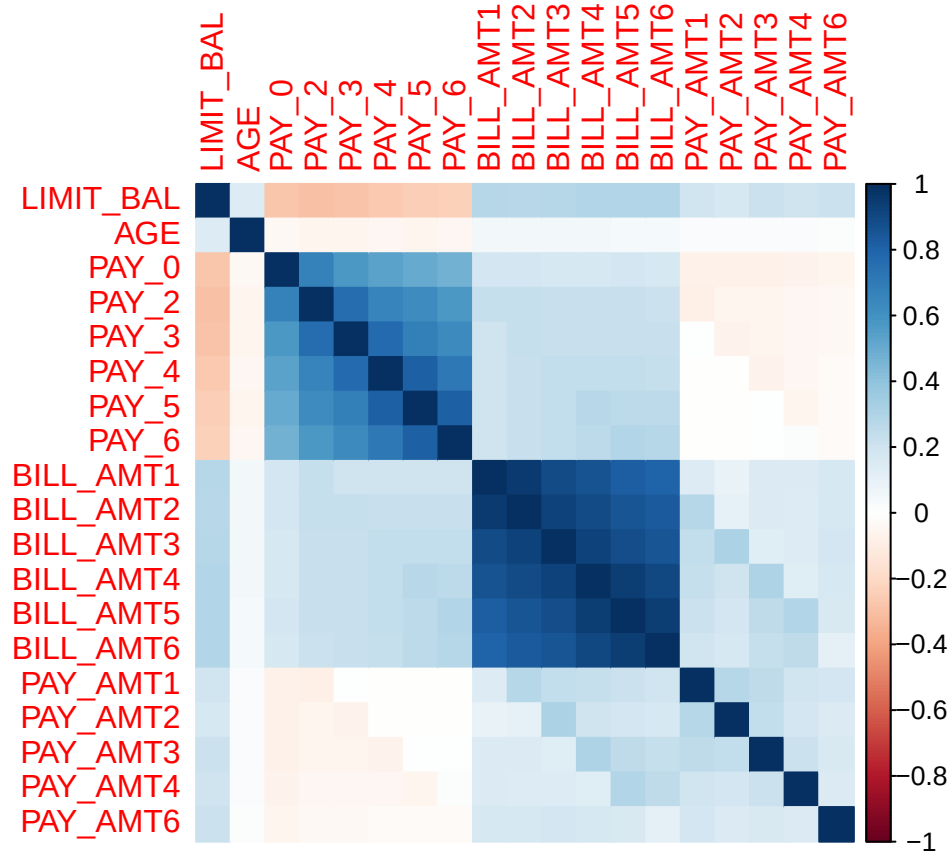


La valeur de Khi-2 est très élevée il y a donc un écart plus important entre les dénombrements observés et attendus.

p est supérieure au niveau de signification choisi, on ne rejette pas l'hypothèse nulle.

Teste de Pearson

```
corrmat <- cor(data[quantitative_variables])
corrplot(corrmat, method = "color")
```



On remarque qu'il n'y a pas de forte relation vraiment entre les variables.

```
# Assuming woebin function exists and returns appropriate data structure
bins <- woebin(data, y = target)
```

```
## i Creating woe binning ...
```

```
## v Binning on 30000 rows and 24 columns in 00:00:13
```

```
bins
```

```
## $LIMIT_BAL
##      variable      bin count count_distr  neg  pos  posprob      woe
## 1: LIMIT_BAL  [-Inf,40000)  4081  0.13603333 2618 1463 0.3584906  0.6767655
## 2: LIMIT_BAL  [40000,140000) 10460 0.34866667 7771 2689 0.2570746  0.1974575
## 3: LIMIT_BAL [140000,380000) 13054 0.43513333 10856 2198 0.1683775 -0.3384831
## 4: LIMIT_BAL [380000, Inf)   2405  0.08016667  2119  286 0.1189189 -0.7440207
##      bin_iv  total_iv breaks is_special_values
## 1: 0.07336909 0.1682805  40000             FALSE
## 2: 0.01433705 0.1682805 140000             FALSE
## 3: 0.04516137 0.1682805 380000             FALSE
## 4: 0.03541303 0.1682805   Inf              FALSE
##
## $SEX
##      variable bin count count_distr  neg  pos  posprob      woe      bin_iv
```

```

## 1:      SEX   1 11888   0.3962667  9015 2873 0.2416723  0.11515393 0.005422772
## 2:      SEX   2 18112   0.6037333 14349 3763 0.2077628 -0.07977674 0.003756807
##      total_iv breaks is_special_values
## 1: 0.009179578      1      FALSE
## 2: 0.009179578      2      FALSE
##
## $EDUCATION
##      variable      bin count count_distr  neg pos  posprob      woe
## 1: EDUCATION      0%,%1 10599   0.3533000 8563 2036 0.1920936 -0.17777649
## 2: EDUCATION      2 14030   0.4676667 10700 3330 0.2373485  0.09141557
## 3: EDUCATION 3%,%4%,%5%,%6 5371   0.1790333 4101 1270 0.2364550  0.08647306
##      bin_iv  total_iv      breaks is_special_values
## 1: 0.010611957 0.01599044      0%,%1      FALSE
## 2: 0.004007548 0.01599044      2      FALSE
## 3: 0.001370935 0.01599044 3%,%4%,%5%,%6      FALSE
##
## $MARRIAGE
##      variable      bin count count_distr  neg pos  posprob      woe
## 1: MARRIAGE 0%,%1 13713      0.4571 10502 3211 0.2341574  0.07370370
## 2: MARRIAGE 2%,%3 16287      0.5429 12862 3425 0.2102904 -0.06448875
##      bin_iv  total_iv breaks is_special_values
## 1: 0.002533998 0.004751178 0%,%1      FALSE
## 2: 0.002217180 0.004751178 2%,%3      FALSE
##
## $AGE
##      variable      bin count count_distr  neg pos  posprob      woe
## 1:      AGE [-Inf,26) 3871   0.1290333 2839 1032 0.2665978  0.24673379
## 2:      AGE  [26,29) 4142   0.1380667 3290 852 0.2056977 -0.09236931
## 3:      AGE  [29,36) 8796   0.2932000 7083 1713 0.1947476 -0.16076432
## 4:      AGE  [36,46) 8522   0.2840667 6661 1861 0.2183760 -0.01646864
## 5:      AGE  [46, Inf) 4669   0.1556333 3491 1178 0.2523024  0.17231686
##      bin_iv  total_iv breaks is_special_values
## 1: 8.389848e-03 0.02169385      26      FALSE
## 2: 1.147627e-03 0.02169385      29      FALSE
## 3: 7.237815e-03 0.02169385      36      FALSE
## 4: 7.668962e-05 0.02169385      46      FALSE
## 5: 4.841874e-03 0.02169385      Inf      FALSE
##
## $PAY_0
##      variable      bin count count_distr  neg pos  posprob      woe
## 1:      PAY_0 [-Inf,0) 8445   0.2815000 7126 1319 0.1561871 -0.4281892
## 2:      PAY_0  [0,1) 14737   0.4912333 12849 1888 0.1281129 -0.6590609
## 3:      PAY_0  [1,2) 3688   0.1229333 2436 1252 0.3394794  0.5930719
## 4:      PAY_0 [2, Inf) 3130   0.1043333 953 2177 0.6955272  2.0847752
##      bin_iv  total_iv breaks is_special_values
## 1: 0.04548860 0.8693808      0      FALSE
## 2: 0.17494106 0.8693808      1      FALSE
## 3: 0.05005818 0.8693808      2      FALSE
## 4: 0.59889299 0.8693808      Inf      FALSE
##
## $PAY_2
##      variable      bin count count_distr  neg pos  posprob      woe
## 1:      PAY_2 [-Inf,-1) 3782   0.1260667 3091 691 0.1827076 -0.2394231
## 2:      PAY_2  [-1,2) 21808   0.7269333 18334 3474 0.1592993 -0.4047636

```

```

## 3:    PAY_2  [2, Inf)  4410   0.1470000  1939 2471 0.5603175  1.5011376
##      bin_iv total_iv breaks is_special_values
## 1: 0.006744204 0.5468569   -1          FALSE
## 2: 0.105725619 0.5468569    2          FALSE
## 3: 0.434387091 0.5468569   Inf          FALSE
##
## $PAY_3
##      variable      bin count count_distr  neg pos  posprob      woe
## 1:    PAY_3  [-Inf,-1)  4085   0.1361667  3328 757 0.1853121 -0.2220765
## 2:    PAY_3   [-1,2)  21706   0.7235333 18028 3678 0.1694462 -0.3308700
## 3:    PAY_3   [2, Inf)  4209   0.1403000  2008 2201 0.5229271  1.3504596
##      bin_iv total_iv breaks is_special_values
## 1: 0.006299561 0.4100696   -1          FALSE
## 2: 0.071919520 0.4100696    2          FALSE
## 3: 0.331850492 0.4100696   Inf          FALSE
##
## $PAY_4
##      variable      bin count count_distr  neg pos  posprob      woe
## 1:    PAY_4  [-Inf,-1)  4348   0.1449333  3511 837 0.1925023 -0.1751451
## 2:    PAY_4   [-1,1)  22142   0.7380667 18222 3920 0.1770391 -0.2778510
## 3:    PAY_4   [1, Inf)  3510   0.1170000  1631 1879 0.5353276  1.4002334
##      bin_iv total_iv breaks is_special_values
## 1: 0.004228655 0.35553    -1          FALSE
## 2: 0.052569554 0.35553     1          FALSE
## 3: 0.298731744 0.35553   Inf          FALSE
##
## $PAY_5
##      variable      bin count count_distr  neg pos  posprob      woe
## 1:    PAY_5  [-Inf,-1)  4546   0.15153333  3651 895 0.1968764 -0.1472457
## 2:    PAY_5   [-1,2)  22486   0.74953333 18394 4092 0.1819799 -0.2443037
## 3:    PAY_5   [2, Inf)  2968   0.09893333  1319 1649 0.5555930  1.4819822
##      bin_iv total_iv breaks is_special_values
## 1: 0.003150416 0.3294371   -1          FALSE
## 2: 0.041688723 0.3294371    2          FALSE
## 3: 0.284597921 0.3294371   Inf          FALSE
##
## $PAY_6
##      variable      bin count count_distr  neg pos  posprob      woe
## 1:    PAY_6  [-Inf,-1)  4895   0.1631667  3914 981 0.2004086 -0.1250557
## 2:    PAY_6   [-1,2)  22026   0.7342000 17982 4044 0.1836012 -0.2334500
## 3:    PAY_6   [2, Inf)  3079   0.1026333  1468 1611 0.5232218  1.3516412
##      bin_iv total_iv breaks is_special_values
## 1: 0.00246268 0.2830788   -1          FALSE
## 2: 0.03740857 0.2830788    2          FALSE
## 3: 0.24320758 0.2830788   Inf          FALSE
##
## $BILL_AMT1
##      variable      bin count count_distr  neg pos  posprob      woe
## 1: BILL_AMT1  [-Inf,10000) 10843   0.3614333 8414 2429 0.2240155  0.01626967
## 2: BILL_AMT1 [10000,20000) 3551   0.1183667 2673 878 0.2472543  0.14537688
## 3: BILL_AMT1 [20000,50000) 6099   0.2033000 4696 1403 0.2300377  0.05058872
## 4: BILL_AMT1 [50000,90000) 3967   0.1322333 3123 844 0.2127552 -0.04970986
## 5: BILL_AMT1 [90000, Inf) 5540   0.1846667 4458 1082 0.1953069 -0.15720205
##      bin_iv total_iv breaks is_special_values

```

```

## 1: 9.610598e-05 0.007911817 10000 FALSE
## 2: 2.602514e-03 0.007911817 20000 FALSE
## 3: 5.276183e-04 0.007911817 50000 FALSE
## 4: 3.222257e-04 0.007911817 90000 FALSE
## 5: 4.363353e-03 0.007911817 Inf FALSE
##
## $BILL_AMT2
## variable bin count count_distr neg pos posprob woe
## 1: BILL_AMT2 [-Inf,5000) 8742 0.2914000 6806 1936 0.2214596 0.001506417
## 2: BILL_AMT2 [5000,10000) 2325 0.0775000 1855 470 0.2021505 -0.114220276
## 3: BILL_AMT2 [10000,35000) 6967 0.2322333 5229 1738 0.2494617 0.157201975
## 4: BILL_AMT2 [35000,90000) 6663 0.2221000 5216 1447 0.2171694 -0.023551373
## 5: BILL_AMT2 [90000, Inf) 5303 0.1767667 4258 1045 0.1970583 -0.146095677
## bin_iv total_iv breaks is_special_values
## 1: 6.615491e-07 0.01071017 5000 FALSE
## 2: 9.788510e-04 0.01071017 10000 FALSE
## 3: 5.989223e-03 0.01071017 35000 FALSE
## 4: 1.223823e-04 0.01071017 90000 FALSE
## 5: 3.619054e-03 0.01071017 Inf FALSE
##
## $BILL_AMT3
## variable bin count count_distr neg pos posprob woe
## 1: BILL_AMT3 [-Inf,10000) 11224 0.3741333 8786 2438 0.2172131 -0.02329451
## 2: BILL_AMT3 [10000,30000) 6439 0.2146333 4873 1566 0.2432055 0.12350184
## 3: BILL_AMT3 [30000,50000) 3575 0.1191667 2744 831 0.2324476 0.06414481
## 4: BILL_AMT3 [50000,90000) 3756 0.1252000 2946 810 0.2156550 -0.03248235
## 5: BILL_AMT3 [90000, Inf) 5006 0.1668667 4015 991 0.1979624 -0.14039109
## bin_iv total_iv breaks is_special_values
## 1: 0.0002016985 0.007377707 10000 FALSE
## 2: 0.0033860246 0.007377707 30000 FALSE
## 3: 0.0004990721 0.007377707 50000 FALSE
## 4: 0.0001309019 0.007377707 90000 FALSE
## 5: 0.0031600103 0.007377707 Inf FALSE
##
## $BILL_AMT4
## variable bin count count_distr neg pos posprob woe
## 1: BILL_AMT4 [-Inf,5000) 9164 0.3054667 7153 2011 0.2194457 -0.01021273
## 2: BILL_AMT4 [5000,15000) 4119 0.1373000 3271 848 0.2058752 -0.09128339
## 3: BILL_AMT4 [15000,35000) 6206 0.2068667 4647 1559 0.2512085 0.16650974
## 4: BILL_AMT4 [35000,65000) 4003 0.1334333 3111 892 0.2228329 0.00945364
## 5: BILL_AMT4 [65000, Inf) 6508 0.2169333 5182 1326 0.2037492 -0.10433719
## bin_iv total_iv breaks is_special_values
## 1: 3.176942e-05 0.009451635 5000 FALSE
## 2: 1.114925e-03 0.009451635 15000 FALSE
## 3: 6.000169e-03 0.009451635 35000 FALSE
## 4: 1.195654e-05 0.009451635 65000 FALSE
## 5: 2.292814e-03 0.009451635 Inf FALSE
##
## $BILL_AMT5
## variable bin count count_distr neg pos posprob woe
## 1: BILL_AMT5 [-Inf,15000) 13826 0.4608667 10858 2968 0.2146680 -0.038326806
## 2: BILL_AMT5 [15000,50000) 8636 0.2878667 6506 2130 0.2466420 0.142084156
## 3: BILL_AMT5 [50000,165000) 6020 0.2006667 4820 1200 0.1993355 -0.131765367
## 4: BILL_AMT5 [165000, Inf) 1518 0.0506000 1180 338 0.2226614 0.008463182

```

```

##          bin_iv  total_iv breaks is_special_values
## 1: 6.697488e-04 0.01006979 15000          FALSE
## 2: 6.040552e-03 0.01006979 50000          FALSE
## 3: 3.355862e-03 0.01006979 165000         FALSE
## 4: 3.632798e-06 0.01006979   Inf          FALSE
##
## $BILL_AMT6
##      variable          bin count count_distr  neg  pos  posprob      woe
## 1: BILL_AMT6  [-Inf,15000) 14297  0.47656667 11267 3030 0.2119326 -0.05462848
## 2: BILL_AMT6  [15000,35000) 6141  0.20470000 4566 1575 0.2564729  0.19430473
## 3: BILL_AMT6  [35000,50000) 2233  0.07443333 1720  513 0.2297358  0.04888328
## 4: BILL_AMT6  [50000, Inf) 7329  0.24430000 5811 1518 0.2071224 -0.08367199
##          bin_iv  total_iv breaks is_special_values
## 1: 0.0014005249 0.01139509 15000          FALSE
## 2: 0.0081438760 0.01139509 35000          FALSE
## 3: 0.0001802852 0.01139509 50000          FALSE
## 4: 0.0016704036 0.01139509   Inf          FALSE
##
## $PAY_AMT1
##      variable          bin count count_distr  neg  pos  posprob      woe
## 1: PAY_AMT1  [-Inf,500) 6650  0.22166667 4452 2198 0.3305263  0.5528814
## 2: PAY_AMT1  [500,5000) 15088 0.50293333 11796 3292 0.2181866 -0.0175782
## 3: PAY_AMT1  [5000,17500) 6676  0.22253333 5683  993 0.1487418 -0.4858169
## 4: PAY_AMT1  [17500, Inf) 1586  0.05286667 1433  153 0.0964691 -0.9784005
##          bin_iv  total_iv breaks is_special_values
## 1: 0.0777760779 0.1608536   500          FALSE
## 2: 0.0001546412 0.1608536  5000          FALSE
## 3: 0.0454720333 0.1608536 17500          FALSE
## 4: 0.0374508406 0.1608536   Inf          FALSE
##
## $PAY_AMT2
##      variable          bin count count_distr  neg  pos  posprob      woe
## 1: PAY_AMT2  [-Inf,500) 6847  0.2282333 4686 2161 0.31561268  0.48467874
## 2: PAY_AMT2  [500,2000) 6756  0.2252000 5176 1580 0.23386619  0.07207929
## 3: PAY_AMT2  [2000,5000) 8436  0.2812000 6646 1790 0.21218587 -0.05311255
## 4: PAY_AMT2  [5000,17500) 6320  0.2106667 5347  973 0.15395570 -0.44521985
## 5: PAY_AMT2  [17500, Inf) 1641  0.0547000 1509  132 0.08043876 -1.17771353
##          bin_iv  total_iv breaks is_special_values
## 1: 0.0606250753 0.1518493   500          FALSE
## 2: 0.0011934767 0.1518493  2000          FALSE
## 3: 0.0007814932 0.1518493  5000          FALSE
## 4: 0.0366112686 0.1518493 17500          FALSE
## 5: 0.0526379564 0.1518493   Inf          FALSE
##
## $PAY_AMT3
##      variable          bin count count_distr  neg  pos  posprob      woe
## 1: PAY_AMT3  [-Inf,500) 7871  0.26236667 5510 2361 0.2999619  0.4112076
## 2: PAY_AMT3  [500,3000) 11124 0.37080000 8610 2514 0.2259978  0.0276378
## 3: PAY_AMT3  [3000,5000) 3859  0.12863333 3124  735 0.1904639 -0.1883120
## 4: PAY_AMT3  [5000,12500) 4951  0.16503333 4173  778 0.1571400 -0.4209770
## 5: PAY_AMT3  [12500, Inf) 2195  0.07316667 1947  248 0.1129841 -0.8019293
##          bin_iv  total_iv breaks is_special_values
## 1: 0.0493258831 0.1166258   500          FALSE
## 2: 0.0002854158 0.1166258  3000          FALSE

```

```
## 3: 0.0043218524 0.1166258 5000 FALSE
## 4: 0.0258348564 0.1166258 12500 FALSE
## 5: 0.0368578094 0.1166258 Inf FALSE
##
## $PAY_AMT4
## variable bin count count_distr neg pos posprob woe
## 1: PAY_AMT4 [-Inf,500) 8775 0.2925000 6285 2490 0.2837607 0.3328039
## 2: PAY_AMT4 [500,2000) 7664 0.2554667 5840 1824 0.2379958 0.0949881
## 3: PAY_AMT4 [2000,4500) 6520 0.2173333 5253 1267 0.1943252 -0.1634604
## 4: PAY_AMT4 [4500,15500) 5481 0.1827000 4613 868 0.1583653 -0.4117550
## 5: PAY_AMT4 [15500, Inf) 1560 0.0520000 1373 187 0.1198718 -0.7349578
## bin_iv total_iv breaks is_special_values
## 1: 0.035351241 0.0931775 500 FALSE
## 2: 0.002365885 0.0931775 2000 FALSE
## 3: 0.005542094 0.0931775 4500 FALSE
## 4: 0.027438865 0.0931775 15500 FALSE
## 5: 0.022479416 0.0931775 Inf FALSE
##
## $PAY_AMT5
## variable bin count count_distr neg pos posprob woe
## 1: PAY_AMT5 [-Inf,500) 8866 0.2955333 6408 2458 0.2772389 0.30048781
## 2: PAY_AMT5 [500,3000) 10781 0.3593667 8313 2468 0.2289213 0.04427456
## 3: PAY_AMT5 [3000,4000) 2394 0.0798000 1920 474 0.1979950 -0.14018614
## 4: PAY_AMT5 [4000,14500) 6237 0.2079000 5189 1048 0.1680295 -0.34097041
## 5: PAY_AMT5 [14500, Inf) 1722 0.0574000 1534 188 0.1091754 -0.84050502
## bin_iv total_iv breaks is_special_values
## 1: 0.0288876214 0.0843597 500 FALSE
## 2: 0.0007131298 0.0843597 3000 FALSE
## 3: 0.0015068801 0.0843597 4000 FALSE
## 4: 0.0218791703 0.0843597 14500 FALSE
## 5: 0.0313728971 0.0843597 Inf FALSE
##
## $PAY_AMT6
## variable bin count count_distr neg pos posprob woe
## 1: PAY_AMT6 [-Inf,1000) 11725 0.39083333 8537 3188 0.2718977 0.27367111
## 2: PAY_AMT6 [1000,3000) 7985 0.26616667 6192 1793 0.2245460 0.01931906
## 3: PAY_AMT6 [3000,4000) 2396 0.07986667 1931 465 0.1940735 -0.16506887
## 4: PAY_AMT6 [4000,10000) 4967 0.16556667 4141 826 0.1662976 -0.35341081
## 5: PAY_AMT6 [10000, Inf) 2927 0.09756667 2563 364 0.1243594 -0.69309285
## bin_iv total_iv breaks is_special_values
## 1: 0.0314772909 0.09031477 1000 FALSE
## 2: 0.0000998752 0.09031477 3000 FALSE
## 3: 0.0020759369 0.09031477 4000 FALSE
## 4: 0.0186480438 0.09031477 10000 FALSE
## 5: 0.0380136210 0.09031477 Inf FALSE
```

```
df <- data.frame(variable = character(), IV = numeric())

# Loop through variables
for(var in colnames(data)) {
  # Check if the bins have any values
  if (length(bins[[var]]$total_iv) > 0) {
    # Access the desired value from bins
    value <- bins[[var]]$total_iv[1]
```

```

    # Add a new row to the data frame
    df <- rbind(df, data.frame(variable = var, IV = value))
  }
}

# Print the dataframe
print(df[df$IV >= 0.3,])

```

```

##      variable      IV
## 6      PAY_0 0.8693808
## 7      PAY_2 0.5468569
## 8      PAY_3 0.4100696
## 9      PAY_4 0.3555300
## 10     PAY_5 0.3294371

```

```

df <- df[df$IV >= 0.3, ]
vars <- df$variable
bins <- bins[vars]
bins

```

```

## $PAY_0
##      variable      bin count count_distr  neg pos  posprob      woe
## 1:      PAY_0 [-Inf,0)  8445   0.2815000  7126 1319 0.1561871 -0.4281892
## 2:      PAY_0  [0,1) 14737   0.4912333 12849 1888 0.1281129 -0.6590609
## 3:      PAY_0  [1,2)  3688   0.1229333  2436 1252 0.3394794  0.5930719
## 4:      PAY_0 [2, Inf)  3130   0.1043333   953 2177 0.6955272  2.0847752
##      bin_iv  total_iv breaks is_special_values
## 1: 0.04548860 0.8693808      0          FALSE
## 2: 0.17494106 0.8693808      1          FALSE
## 3: 0.05005818 0.8693808      2          FALSE
## 4: 0.59889299 0.8693808     Inf          FALSE
##
## $PAY_2
##      variable      bin count count_distr  neg pos  posprob      woe
## 1:      PAY_2 [-Inf,-1)  3782   0.1260667  3091  691 0.1827076 -0.2394231
## 2:      PAY_2  [-1,2) 21808   0.7269333 18334 3474 0.1592993 -0.4047636
## 3:      PAY_2  [2, Inf)  4410   0.1470000  1939 2471 0.5603175  1.5011376
##      bin_iv  total_iv breaks is_special_values
## 1: 0.006744204 0.5468569     -1          FALSE
## 2: 0.105725619 0.5468569      2          FALSE
## 3: 0.434387091 0.5468569     Inf          FALSE
##
## $PAY_3
##      variable      bin count count_distr  neg pos  posprob      woe
## 1:      PAY_3 [-Inf,-1)  4085   0.1361667  3328  757 0.1853121 -0.2220765
## 2:      PAY_3  [-1,2) 21706   0.7235333 18028 3678 0.1694462 -0.3308700
## 3:      PAY_3  [2, Inf)  4209   0.1403000  2008 2201 0.5229271  1.3504596
##      bin_iv  total_iv breaks is_special_values
## 1: 0.006299561 0.4100696     -1          FALSE
## 2: 0.071919520 0.4100696      2          FALSE
## 3: 0.331850492 0.4100696     Inf          FALSE
##

```

```
## $PAY_4
##   variable      bin count count_distr  neg pos  posprob      woe
## 1:   PAY_4 [-Inf,-1)  4348   0.1449333 3511  837 0.1925023 -0.1751451
## 2:   PAY_4  [-1,1)  22142   0.7380667 18222 3920 0.1770391 -0.2778510
## 3:   PAY_4  [1, Inf)   3510   0.1170000 1631 1879 0.5353276  1.4002334
##       bin_iv total_iv breaks is_special_values
## 1: 0.004228655  0.35553      -1          FALSE
## 2: 0.052569554  0.35553       1          FALSE
## 3: 0.298731744  0.35553      Inf          FALSE
##
## $PAY_5
##   variable      bin count count_distr  neg pos  posprob      woe
## 1:   PAY_5 [-Inf,-1)  4546   0.15153333 3651  895 0.1968764 -0.1472457
## 2:   PAY_5  [-1,2)  22486   0.74953333 18394 4092 0.1819799 -0.2443037
## 3:   PAY_5  [2, Inf)   2968   0.09893333 1319 1649 0.5555930  1.4819822
##       bin_iv total_iv breaks is_special_values
## 1: 0.003150416  0.3294371      -1          FALSE
## 2: 0.041688723  0.3294371       2          FALSE
## 3: 0.284597921  0.3294371      Inf          FALSE
```

```
data <- data[c(vars, "default.payment.next.month")]
```

```
data$PAY_0[data$PAY_0 < 2] <- 0
data$PAY_0[data$PAY_0 >= 2] <- 1
```

```
data$PAY_2[data$PAY_0 < 2] <- 0
data$PAY_2[data$PAY_0 >= 2] <- 1
```

```
data$PAY_3[data$PAY_3 < 2] <- 0
data$PAY_3[data$PAY_3 >= 2] <- 1
```

```
data$PAY_4[data$PAY_4 < 1] <- 0
data$PAY_4[data$PAY_4 >= 1] <- 1
```

```
data$PAY_5[data$PAY_5 < 2] <- 0
data$PAY_5[data$PAY_5 >= 2] <- 1
```

```
data$PAY_0 <- factor(data$PAY_0)
data$PAY_2 <- factor(data$PAY_2)
data$PAY_3 <- factor(data$PAY_3)
data$PAY_4 <- factor(data$PAY_4)
data$PAY_5 <- factor(data$PAY_5)
```

```
cramers_v <- function(table) {
  chi_square <- chisq.test(table)$statistic
  n <- sum(table)
  min_dim <- min(dim(table))

  if (min_dim - 1 == 0) {
    return(0) # or any other suitable value
  }

  phi <- sqrt(chi_square / (n * (min_dim - 1)))
}
```

```

cramer_v <- phi / sqrt(min_dim - 1)
return(cramer_v)
}

```

```

cramers_v_list <- list()
n_vars <- length(vars)
cramers_v_matrix <- matrix(NA, nrow = n_vars, ncol = n_vars, dimnames = list(vars, vars))
for(var1 in vars){
  for(var2 in vars[vars != var1]){
    cont_table <- table(data[[var1]], data[[var2]])
    v <- cramers_v(cont_table)
    cramers_v_list <- c(cramers_v_list, paste(var1, var2, v))
  }
}
cramers_v_list

```

```

## [[1]]
## [1] "PAY_0 PAY_2 0"
##
## [[2]]
## [1] "PAY_0 PAY_3 0.412673292752333"
##
## [[3]]
## [1] "PAY_0 PAY_4 0.383112390279739"
##
## [[4]]
## [1] "PAY_0 PAY_5 0.364787175058505"
##
## [[5]]
## [1] "PAY_2 PAY_0 0"
##
## [[6]]
## [1] "PAY_2 PAY_3 0"
##
## [[7]]
## [1] "PAY_2 PAY_4 0"
##
## [[8]]
## [1] "PAY_2 PAY_5 0"
##
## [[9]]
## [1] "PAY_3 PAY_0 0.412673292752333"
##
## [[10]]
## [1] "PAY_3 PAY_2 0"
##
## [[11]]
## [1] "PAY_3 PAY_4 0.625003061082114"
##
## [[12]]
## [1] "PAY_3 PAY_5 0.482218495314962"
##
## [[13]]

```

```
## [1] "PAY_4 PAY_0 0.383112390279739"
##
## [[14]]
## [1] "PAY_4 PAY_2 0"
##
## [[15]]
## [1] "PAY_4 PAY_3 0.625003061082114"
##
## [[16]]
## [1] "PAY_4 PAY_5 0.662463472835302"
##
## [[17]]
## [1] "PAY_5 PAY_0 0.364787175058505"
##
## [[18]]
## [1] "PAY_5 PAY_2 0"
##
## [[19]]
## [1] "PAY_5 PAY_3 0.482218495314962"
##
## [[20]]
## [1] "PAY_5 PAY_4 0.662463472835302"
```

```
data <- subset(data, select = -PAY_4)
data <- subset(data, select = -PAY_2)
head(data)
```

```
##   PAY_0 PAY_3 PAY_5 default.payment.next.month
## 1     1     0     0                        1
## 2     0     0     0                        1
## 3     0     0     0                        0
## 4     0     0     0                        0
## 5     0     0     0                        0
## 6     0     0     0                        0
```

```
# Load the caret package
library(caret)
```

```
## Warning: package 'caret' was built under R version 4.3.3
```

```
## Loading required package: lattice
```

```
# Set the seed for reproducibility
set.seed(123)
```

```
# Assuming 'data' is your dataframe and 'target_variable' is your target variable
# Create an index for splitting the data
```

```
train_index <- createDataPartition(data$default.payment.next.month, p = 0.7, list = FALSE)
```

```
# Split the data into training and testing sets
```

```
train_data <- data[train_index, ]
test_data <- data[-train_index, ]
```

```

X_train <- subset(train_data, select = -default.payment.next.month)
y_train <- train_data$default.payment.next.month
X_test <- subset(test_data, select = -default.payment.next.month)
y_test <- test_data$default.payment.next.month

# Fit a logistic regression model
logistic_model <- glm(default.payment.next.month ~ ., data = train_data, family = binomial)

probabilities <- predict(logistic_model, X_test, type = "response")
corrected_predictions <- ifelse(probabilities < 0.5, 0, 1)

confusion_matrix <- table(corrected_predictions, y_test)
confusion_matrix

##               y_test
## corrected_predictions  0    1
##                   0 6752 1328
##                   1  291  629

TP <- confusion_matrix[1,1]
FP <- confusion_matrix[1,2]
FN <- confusion_matrix[2,1]
TN <- confusion_matrix[2,2]

Accuracy <- (TP + TN) / (TP + TN + FP + FN)
Precision <- TP / (TP + FP)
Recall <- TP / (TP + FN)
Specificity <- TN / (TN + FP)
F1Score <- 2 * (Precision * Recall) / (Precision + Recall)

Accuracy

## [1] 0.8201111

Precision

## [1] 0.8356436

Recall

## [1] 0.9586824

Specificity

## [1] 0.3214103

F1Score

## [1] 0.8929445

```

```
# Assuming 'true_labels' are the true labels and 'predicted_scores' are the predicted probabilities or  
library(pROC)
```

```
## Warning: package 'pROC' was built under R version 4.3.3
```

```
## Type 'citation("pROC")' for a citation.
```

```
##
```

```
## Attaching package: 'pROC'
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
##      cov, smooth, var
```

```
# Calculate AUC
```

```
auc <- auc(roc(y_test, corrected_predictions))
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
print(auc)
```

```
## Area under the curve: 0.64
```

```
roc_curve <- roc(y_test, corrected_predictions)
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
# Plot ROC curve
```

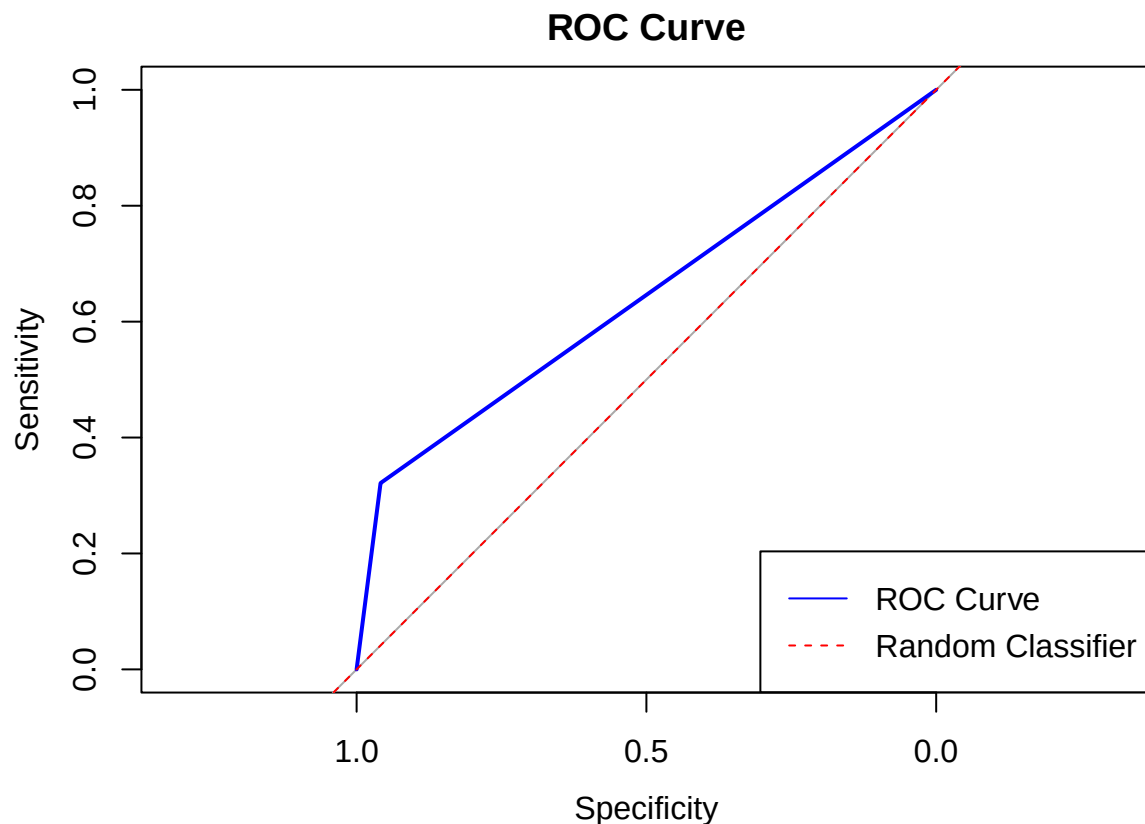
```
plot(roc_curve, main = "ROC Curve", col = "blue")
```

```
# Add diagonal line (random classifier)
```

```
abline(a = 1, b = -1, lty = 2, col = "red")
```

```
# Add legend
```

```
legend("bottomright", legend = c("ROC Curve", "Random Classifier"),  
      col = c("blue", "red"), lty = c(1, 2))
```



Random Forest Classifier

```
rf_model <- randomForest(x = X_train, y = y_train, ntree = 1000)
```

```
## Warning in randomForest.default(x = X_train, y = y_train, ntree = 1000): The
## response has five or fewer unique values. Are you sure you want to do
## regression?
```

```
probabilities_rf <- predict(rf_model, X_test)
corrected_predictions_rf <- ifelse(probabilities < 0.5, 0, 1)
```

```
confusion_matrix_rf <- table(corrected_predictions_rf, y_test)
confusion_matrix_rf
```

```
##               y_test
## corrected_predictions_rf    0    1
##               0 6752 1328
##               1  291  629
```

```
TP_rf <- confusion_matrix_rf[1,1]
FP_rf <- confusion_matrix_rf[1,2]
FN_rf <- confusion_matrix_rf[2,1]
TN_rf <- confusion_matrix_rf[2,2]
```

```
Accuracy_rf <- (TP_rf + TN_rf) / (TP_rf + TN_rf + FP_rf + FN_rf)
Precision_rf <- TP_rf / (TP_rf + FP_rf)
Recall_rf <- TP_rf / (TP_rf + FN_rf)
Specificity_rf <- TN_rf / (TN_rf + FP_rf)
F1Score_rf <- 2 * (Precision_rf * Recall_rf) / (Precision_rf + Recall_rf)
```

```
# Calculate AUC
auc <- auc(roc(y_test, corrected_predictions_rf))
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
print(auc)
```

```
## Area under the curve: 0.64
```